REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
ÉCOLE SUPÉRIEURE PRIVÉE
DE TECHNOLOGIES & INGÉNIERIE

Rapport de projet Développement

Cycle de Formation des Ingénieurs en Informatique

SaveSafe Wallet SSW

Réalisé par :

AMOR Houssem, KHEDHIRI Montaha,
BEN SLIMA Wissem, BAHROUNI Seif

Encadré par :

Ines AGREBI

Année Universitaire : 2025-2026

Table des matières

Introduction Générale **3**

I Présentation générale du projet **4**
I.1 Contexte 4
I.2 Objectifs du projet 4
I.2.1 Objectif général 4
I.2.2 Objectifs spécifiques 4

II Description de l'idée **6**
II.1 Nom du projet 6
II.2 Description générale du projet 6
II.3 Problématique 7
II.3.1 Situation actuelle 7
II.3.2 Problèmes rencontrés 8
II.3.3 Solution proposée 8

III Étude de l'état de l'art (benchmark) **9**
III.1 Étude comparative 9
III.2 Valeur ajoutée de SaveSafe Wallet 9

IV Spécification des besoins **11**
IV.1 Besoins fonctionnels 11
IV.1.1 Tableau des besoins fonctionnels 12
IV.2 Besoins non fonctionnels 13

V Spécification des acteurs **14**
V.1 Liste des acteurs 14
V.2 Description des acteurs 14

VI Diagramme de cas d'utilisation **16**
VI.1 Diagramme de cas d'utilisation – Visiteur + Utilisateur 17
VI.2 Diagramme de cas d'utilisation – Administrateur 18
VI.3 Diagramme de cas d'utilisation – AI 19

VII Gestion de projet SCRUM **20**

VII.1 Backlog du produit (Product Backlog) 20
VII.2 Planification des Sprints / Releases 23
VII.3 Répartition des rôles SCRUM 23
VII.4 Choix Technologiques et Outils de Collaboration 23
VII.5 Développement du Sprint 1 (MVP) 24
VII.5.1 Objectif Sprint 1 24
VII.5.2 Tâches Sprint 1 24
VII.5.3 Livrables Sprint 1 24

Conclusion Générale **25**

Table des figures

VI.1 Diagramme de cas d'utilisation – Visiteur + Utilisateur 17
VI.2 Diagramme de cas d'utilisation – Administrateur 18
VI.3 Diagramme de cas d'utilisation – AI 19

Liste des tableaux

III.1 Étude comparative des solutions existantes 9

IV.1 Besoins fonctionnels détaillés (BF1–BF17) 12
IV.2 Besoins non fonctionnels 13

V.1 Liste des acteurs 14

VII.1 Product Backlog 22
VII.2 Planification des Sprints / Releases 23
VII.3 Rôles SCRUM 23
VII.4 Tâches Sprint 1 24

Introduction Générale

Le présent rapport s'inscrit dans le cadre du module *Projet de Développement* du cycle de formation des ingénieurs en informatique à l'École Supérieure Privée de Technologies et d'Ingénierie (TEK-UP), pour l'année universitaire 2025-2026.

Le projet **SaveSafe Wallet (SSW)** consiste à concevoir et développer une plateforme web bancaire sécurisée, intégrant un portefeuille numérique (*wallet*) et un système anti-piratage propulsé par l'Intelligence Artificielle. L'objectif est de fournir aux utilisateurs finaux une expérience bancaire moderne, tout en offrant aux administrateurs un tableau de bord de sécurité en temps réel alimenté par des modèles d'IA open-source (Ollama, Isolation Forest, Autoencoders).

L'architecture repose sur un backend .NET 8 organisé en microservices (Auth, User, Wallet, Payment, Security, AI Risk, Notification, Reporting), un frontend Angular, et une couche d'observabilité (Grafana, Prometheus, Loki). La communication asynchrone est assurée par Apache Kafka avec un patron Outbox pour la fiabilité.

La méthodologie Agile SCRUM est adoptée pour organiser le développement en cinq sprints de deux semaines chacun, avec des livraisons progressives depuis le MVP jusqu'à la release de production.

Ce rapport présente successivement le cadre général du projet, la description de l'idée, l'étude de l'état de l'art, la spécification des besoins et acteurs, les diagrammes de cas d'utilisation, et la gestion de projet SCRUM.

I – Présentation générale du projet

I.1 Contexte

Dans le cadre du module *Projet de Développement* du cycle ingénieur en informatique à TEK-UP, encadré par Mme Ines AGREBI, ce projet vise à mettre en pratique les connaissances acquises en :

- Développement web full-stack (Angular + .NET 8)
- Intelligence Artificielle (détection d'anomalies, scoring de risques)
- Bases de données relationnelles et cache (PostgreSQL, Redis)
- Infrastructure cloud et conteneurs (Docker, Kubernetes, AWS)
- Méthodologie Agile SCRUM

Le projet **SaveSafe Wallet (SSW)** consiste à concevoir et développer une plateforme web bancaire sécurisée, combinant un portefeuille numérique (*wallet*), un module de paiement via Stripe, et un système anti-piratage propulsé par l'IA. La méthodologie SCRUM est adoptée pour organiser le travail en sprints de deux semaines, assurer la communication au sein de l'équipe et livrer progressivement un produit fonctionnel.

I.2 Objectifs du projet

I.2.1 Objectif général

Développer une plateforme bancaire web sécurisée avec portefeuille numérique, intégrant un système de détection de menaces par Intelligence Artificielle et un tableau de bord d'administration en temps réel.

I.2.2 Objectifs spécifiques

L'application SaveSafe Wallet vise à :

- Offrir aux utilisateurs la gestion de comptes, wallets et transferts pair-à-pair
- Intégrer un module de paiement sécurisé via Stripe (checkout, webhooks, réconciliation)
- Détecter automatiquement les comportements suspects (tentatives de connexion, injections SQL, anomalies comportementales) grâce à des modèles d'IA open-source
- Fournir aux administrateurs un tableau de bord Grafana affichant les attaques, IP bloquées, et métriques système en temps réel

- Assurer une communication asynchrone fiable entre microservices via Apache Kafka (patron Outbox)
- Générer des rapports de sécurité et des KPIs métier (volume de transactions, taux de succès/échecs)

II – Description de l'idée

II.1 Nom du projet

Le projet est nommé **SaveSafe Wallet (SSW)**. Ce nom reflète la double vocation de la plateforme : offrir un portefeuille numérique (*Wallet*) et garantir un haut niveau de sécurité (*SaveSafe*) grâce à l'Intelligence Artificielle.

II.2 Description générale du projet

SaveSafe Wallet est une plateforme web bancaire sécurisée organisée en microservices. Elle couvre les modules fonctionnels suivants :

Gestion des comptes et authentification

- Inscription et authentification sécurisée (JWT, refresh tokens)
- Authentification Google OAuth 2.0
- Gestion des profils utilisateurs et préférences
- Contrôle d'accès par rôles RBAC (Utilisateur, Administrateur)

Portefeuille numérique (Wallet) et Ledger

- Création automatique d'un wallet à l'inscription
- Consultation du solde en temps réel
- Ledger immuable enregistrant chaque opération (crédit/débit)
- Historique des mouvements avec filtres et pagination

Paiement et transactions (Stripe)

- Checkout sécurisé via Stripe (create-checkout-session)
- Gestion des webhooks Stripe (succès, échec)
- Réconciliation et idempotency pour éviter les doublons
- Publication d'événements de paiement via Kafka

Sécurité et détection de menaces (IA)

- Collecte des événements de sécurité (login, IP, user-agent)
- Détection d'anomalies comportementales (Isolation Forest, Autoencoders)
- Scoring de risque en temps réel via un service IA asynchrone (Ollama + Mistral/LLaMA)
- Détection d'injections SQL (règles + modèle ML)
- Rate-limiting Redis et blocklist IP automatique
- Audit trail immuable avec chaînage de hash

Notification et reporting

- Notifications transactionnelles par email (paiement réussi/échoué)
- Alertes de sécurité pour les administrateurs
- Rapports agrégés (KPIs paiement, volume wallet, incidents sécurité)

Tableau de bord administrateur (Grafana)

- Visualisation des attaques par jour/heure
- IP bloquées et géolocalisation
- Tentatives de connexion échouées
- Détections d'injections SQL
- Métriques système (CPU, RAM, latence)

Interface utilisateur

- Application Angular responsive (PC / mobile)
- Dashboard utilisateur (comptes, solde, historique, transferts)
- Console d'administration (sécurité, analytics, gestion utilisateurs)
- Design moderne avec navigation intuitive

II.3 Problématique

II.3.1 Situation actuelle

Les plateformes bancaires et fintech actuelles, notamment dans le contexte tunisien, présentent plusieurs faiblesses :

- Sécurité réactive – les menaces sont détectées après les dégâts, pas en amont
- Surveillance manuelle des logs et événements de sécurité par les équipes IT
- Absence d'IA intégrée pour la détection d'anomalies et le scoring de risque
- Interfaces d'administration limitées, sans visualisation en temps réel
- Coûts élevés des solutions de cybersécurité avancées (Darktrace, CrowdStrike)
- Dépendance à des APIs payantes (OpenAI, etc.) pour les fonctionnalités intelligentes

II.3.2 Problèmes rencontrés

- Vulnérabilité des plateformes face aux cyberattaques (brute force, injection SQL, bots)
- Difficulté à surveiller les activités de connexion et transactions en temps réel
- Absence de tableau de bord centralisé pour les administrateurs de sécurité
- Pas de corrélation automatique entre événements suspects (voyage impossible, changement de device, pic de transactions)

II.3.3 Solution proposée

SaveSafe Wallet répond à ces problèmes en :

- Intégrant des modèles d'IA open-source (Ollama, Isolation Forest) pour une détection proactive des menaces
- Automatisant l'analyse des événements de sécurité via un pipeline Kafka asynchrone
- Offrant un tableau de bord Grafana en temps réel pour les administrateurs (attaques, IP, métriques)
- Sécurisant les transactions via Stripe avec réconciliation et audit trail immuable
- Proposant une architecture microservices .NET 8 modulaire et évolutive
- Utilisant uniquement des outils gratuits et open-source pour rester accessible

III – Étude de l'état de l'art (benchmark)

III.1 Étude comparative

Solution	Fonction	IA	Sécurité	Limites
Revolut	Banque mobile, wallet multi-devises	Partielle (fraude)	Élevée (3DS, biométrie)	Code source fermé, pas d'accès aux modèles IA
PayPal	Paiement en ligne	Basique (détection fraude)	Élevée (chiffrement)	Pas de dashboard sécurité admin, peu d'IA visible
Wise	Transferts internationaux	Minimale	Bonne (régulé FCA)	Focalisé transferts, pas de détection de menaces
Darktrace	Cybersécurité entreprise	Très avancée (self-learning)	Excellente	Très coûteux, non adapté aux petites structures
Flouci	Paiement mobile (Tunisie)	Aucune	Basique	Pas d'IA, pas de dashboard sécurité, fonctionnalités limitées
N26	Néobanque européenne	Partielle	Élevée	Fermée, pas de monitoring sécurité admin accessible

Table III.1 – Étude comparative des solutions existantes

III.2 Valeur ajoutée de SaveSafe Wallet

IA open-source intégrée

- Modèles Ollama (Mistral, LLaMA) gratuits et hébergés localement – aucune dépendance à des APIs payantes
- Scoring de risque en temps réel et détection d'anomalies comportementales
- Détection d'injections SQL par approche hybride (règles + ML)

Tableau de bord sécurité dédié

- Dashboard Grafana en temps réel pour les administrateurs (attaques, IP bloquées, métriques)

- Contrairement aux solutions concurrentes (Revolut, PayPal, Flouci), l'admin a une visibilité complète
- Alertes configurables avec seuils personnalisables

Architecture microservices moderne

- Backend .NET 8 découpé en services indépendants (Auth, Wallet, Payment, Security, AI Risk, etc.)
- Communication asynchrone fiable via Kafka avec patron Outbox
- Conteneurisation Docker et orchestration Kubernetes

Adaptation au contexte tunisien

- Solution entièrement gratuite et open-source, accessible pour le marché local
- Interface en français
- Alternative aux solutions coûteuses (Darktrace) ou fermées (Revolut, N26)

Évolutivité

- Architecture cloud-ready (AWS free tier, Kubernetes scaling)
- Modules indépendants extensibles (ajout de services sans impact sur l'existant)
- Modèles IA remplaçables et améliorables sans refactoring du code métier

IV – Spécification des besoins

IV.1 Besoins fonctionnels

Côté visiteur

- BF1 : Consulter les pages publiques
- BF2 : Voir les fonctionnalités
- BF3 : Créer un compte

Côté utilisateur

- BF4 : S'inscrire
- BF5 : Se connecter
- BF6 : Modifier profil
- BF7 : Consulter données
- BF8 : Recevoir alertes
- BF9 : Consulter historique

Côté admin

- BF10 : Gérer utilisateurs
- BF11 : Voir statistiques
- BF12 : Configurer IA
- BF13 : Bloquer comptes
- BF14 : Générer rapports

Côté IA

- BF15 : Analyser données
- BF16 : Détecter anomalies
- BF17 : Prédire incidents

IV.1.1 Tableau des besoins fonctionnels

ID	Besoin	Description
BF1	Consulter pages publiques	Le visiteur peut consulter la page d'accueil et les fonctionnalités sans inscription
BF2	Voir les fonctionnalités	Le visiteur visualise les services proposés par la plateforme
BF3	Créer un compte	Le visiteur peut s'inscrire pour devenir utilisateur
BF4	S'inscrire	Création de compte avec email, mot de passe et validation
BF5	Se connecter	Authentification sécurisée via JWT (login/password ou Google OAuth)
BF6	Modifier profil	L'utilisateur peut mettre à jour ses informations personnelles et préférences
BF7	Consulter données	Consultation du solde wallet, des comptes et des transactions
BF8	Recevoir alertes	Notifications automatiques (email) pour paiements et événements de sécurité
BF9	Consulter historique	Affichage de l'historique des mouvements wallet avec filtres et pagination
BF10	Gérer utilisateurs	L'administrateur peut lister, activer, désactiver et bloquer des comptes
BF11	Voir statistiques	Consultation du tableau de bord Grafana (attaques, KPIs, métriques système)
BF12	Configurer IA	L'administrateur ajuste les seuils de scoring IA et les politiques de réponse
BF13	Bloquer comptes	Blocage manuel ou automatique de comptes jugés suspects
BF14	Générer rapports	Génération de rapports agrégés (paiements, sécurité, wallet)
BF15	Analyser données	Le système IA analyse les événements de sécurité en temps réel
BF16	Détecter anomalies	Détection automatique d'anomalies comportementales (Isolation Forest, Autoencoders)
BF17	Prédire incidents	Scoring de risque et prédiction de menaces via modèles Ollama

Table IV.1: Besoins fonctionnels détaillés (BF1–BF17)

IV.2 Besoins non fonctionnels

Critère	**Exigence**
Sécurité	SSL, JWT, chiffrement
Performance	< 2 secondes
Disponibilité	99%
Scalabilité	Cloud
Ergonomie	Responsive
Maintenance	Code modulaire
Fiabilité	Tolérance aux pannes

Table IV.2 – Besoins non fonctionnels

V – Spécification des acteurs

V.1 Liste des acteurs

Acteur	Rôle
Visiteur	Consulte l'application sans inscription
Utilisateur	Utilise les services après inscription
Administrateur	Gère le système
Système IA	Analyse et prédit automatiquement

Table V.1 – Liste des acteurs

V.2 Description des acteurs

Visiteur

- Accède à la page d'accueil
- Consulte les fonctionnalités
- Visualise les offres
- Décide de créer un compte

Utilisateur

- Crée un compte
- Se connecte
- Utilise les services
- Consulte ses données
- Reçoit des alertes
- Gère son profil

Administrateur

- Gère les utilisateurs
- Supervise la sécurité

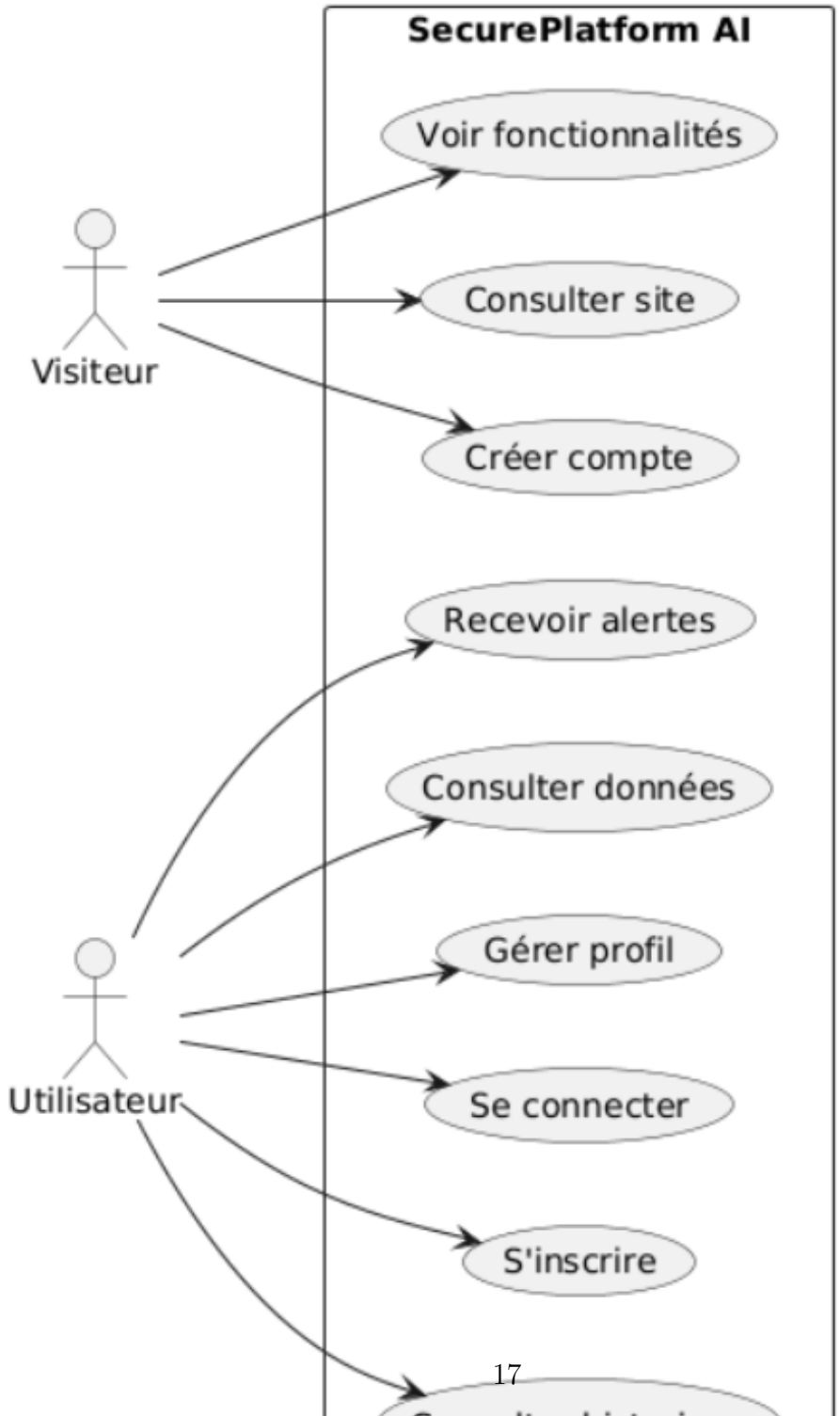
- Consulte les statistiques
- Configure l'IA
- Génère les rapports

Système IA

- Analyse les données
- Détecte les anomalies
- Prédit les risques
- Génère des alertes
- Apprend automatiquement

VI – Diagramme de cas d'utilisation

VI.1 Diagramme de cas d'utilisation – Visiteur + Utilisateur



VI.2 Diagramme de cas d'utilisation – Administrateur

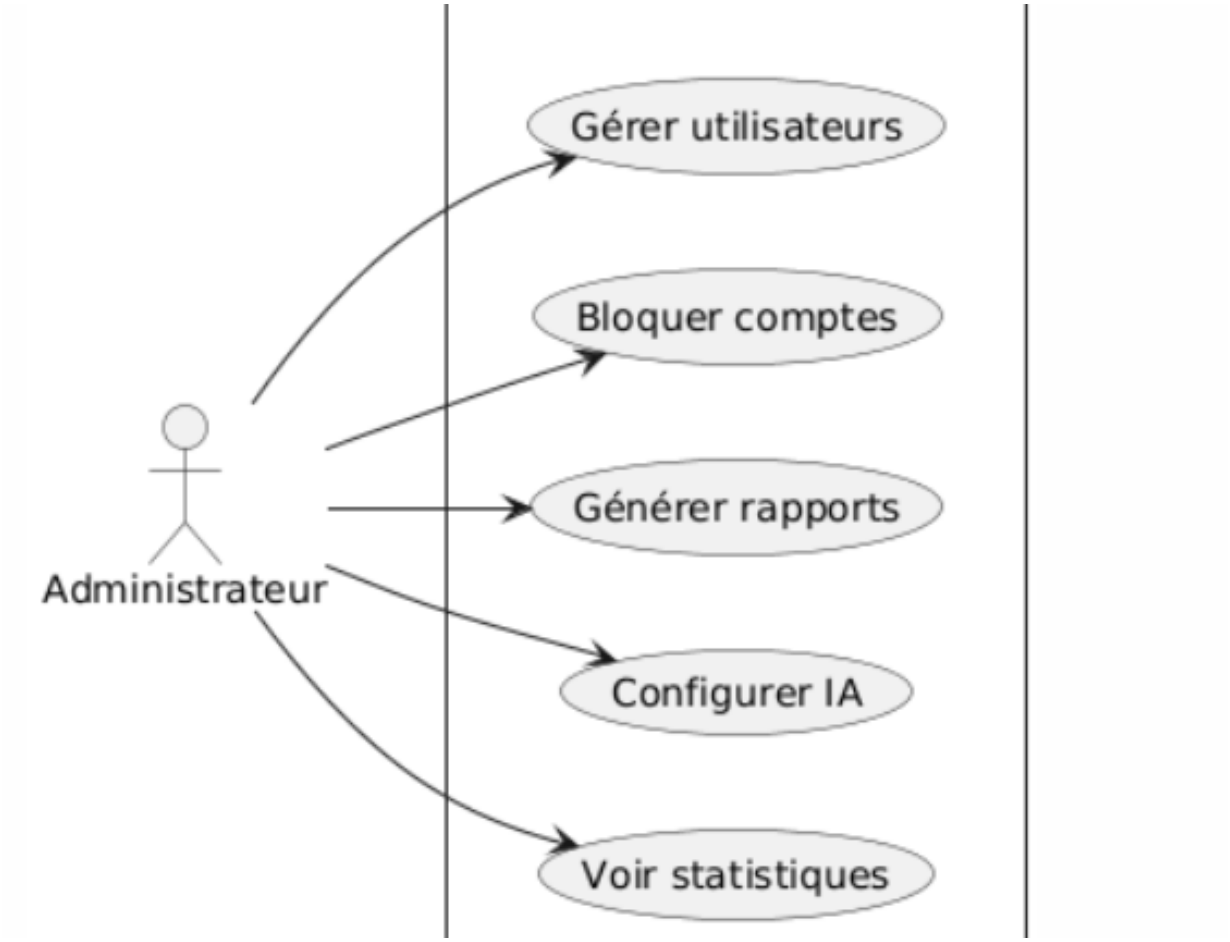


FIGURE VI.2 – Diagramme de cas d'utilisation – Administrateur

VI.3 Diagramme de cas d'utilisation – AI

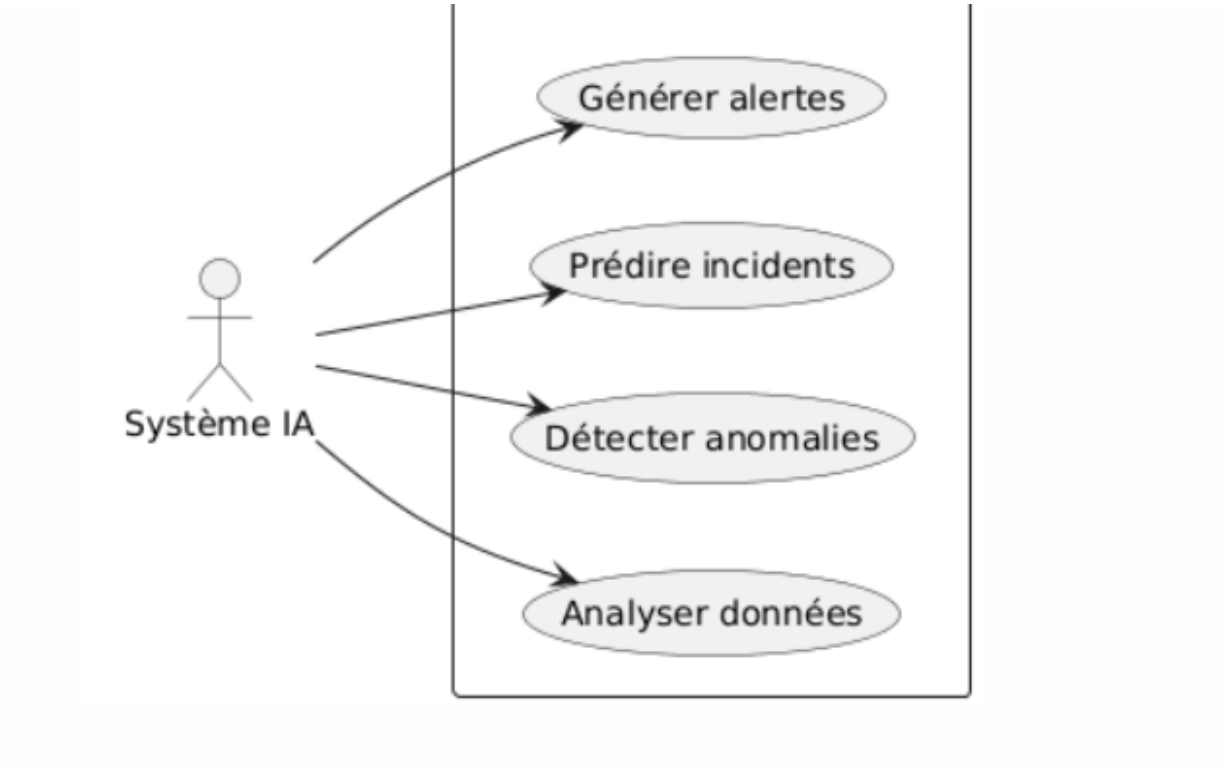


FIGURE VI.3 – Diagramme de cas d'utilisation – AI

VII – Gestion de projet SCRUM

VII.1 Backlog du produit (Product Backlog)

Le backlog reprend les EPIC, Stories et Tasks et sert de base pour la méthodologie SCRUM.

EPIC	Story	Task
E1 — Cadrage & Architecture	E1-S1 — Définir le périmètre MVP	Lister fonctionnalités MVP in-scope (Stripe only, sans APIs opérateurs) Lister out-of-scope (intégrations CNAM/STEG/Orange réelles) Valider user stories prioritaires (User + Admin)
	E1-S2 — Finaliser architecture backend	Valider services finaux (Gateway, Auth, User, Wallet, Payment, Security, Notification, AI Risk, Reporting) Définir communication Sync/Async Définir topics Kafka v1
E2 — Setup Projet & DevOps	E2-S1 — Initialiser repos et standards	Créer structure mono-repo ou multi-repo Ajouter conventions branches (main/dev/feature/*) Ajouter templates PR + règles review
	E2-S2 — Environnement local	Docker Compose (PostgreSQL, Redis, Kafka, Grafana, Loki) Ajouter health checks de chaque service Documenter README démarrage local
	E2-S3 — CI/CD basique	Pipeline build + test unitaire Scan qualité (lint + format) Artifacts Docker images
E3 — Auth & User	E3-S1 — Auth JWT	Endpoint register/login Refresh token + logout Middleware RBAC (User, Admin)
	E3-S2 — Profil utilisateur	CRUD profil

EPIC	Story	Task
		Préférences utilisateur (langue, notif) Journaliser actions sensibles
E4 — Wallet & Ledger	E4-S1 — Gestion wallet E4-S2 — Historique wallet	Créer wallet à l'inscription Endpoint consultation solde Ledger immuable (credit/debit) Endpoint historique mouvements Filtre date/statut/type Pagination + tri
E5 — Paiement Stripe	E5-S1 — Checkout Stripe E5-S2 — Webhook Stripe E5-S3 — Réconciliation basique	Endpoint create-checkout-session Mapper serviceType statique + montant Sauvegarder transaction PENDING Vérifier signature webhook Mettre transaction SUCCESS/FAILED Publier événement Kafka paiement Idempotency key anti double paiement Retry sécurisé en cas d'échec interne Rapport journalier paiements
E6 — Security Service	E6-S1 — Collecte logs sécurité E6-S2 — Règles temps réel E6-S3 — Audit immuable	Capturer login success/failed, IP, user-agent Stocker security_events Ajouter correlation_id partout Rate-limit Redis Blocklist IP (manual + auto rules) Détection patterns SQLi basiques Table append-only admin actions Chaînage hash (prev_hash, curr_hash) Endpoint timeline sécurité admin
E7 — Asynchrone Kafka (fiabilité)	E7-S1 — Outbox pattern E7-S2 — Consumers robustes	Table outbox_events Worker de publication Kafka Marquage PENDING/SENT/FAILED Idempotent consumer via event_id

EPIC	Story	Task
		Retry policy Dead-letter topic (DLT)
E8 — Notification Service	E8-S1 — Notifications transactionnelles E8-S2 — Fiabilité envoi	Template email paiement réussi Template email paiement échoué Consommation événement payment.* Retry envoi + backoff Statuts sent/failed Journal notifications
E9 — AI Risk Service (Async)	E9-S1 — Scoring risque E9-S2 — Intégration Security	Consommer security.event.created Calcul score (règles + modèle) Publier risk.score.computed Action policy ALLOW/CHALLENGE/TEMP_BLOCK Alertes admin sur seuil critique Fallback si IA down (règles only)
E10 — Reporting & Monitoring	E10-S1 — Reporting métier E10-S2 — Observabilité	Agrégats paiements (jour/semaine/mois) KPIs wallet (volume, succès, échecs) Endpoint dashboard admin Metrics Prometheus par service Dashboards Grafana (paiement, sécurité, infra) Logs centralisés Loki
E11 — Tests & Hardening	E11-S1 — Tests E11-S2 — Sécurité production	Unit tests services critiques Integration tests (Stripe webhook flow) E2E tests (login → paiement → notif) CORS strict + headers sécurité Secrets management Vérification conformité logs (pas de PII sensible)

Table VII.1: Product Backlog

VII.2 Planification des Sprints / Releases

Sprint	Durée	Contenu principal	Objectif Release
Sprint 1	2 semaines	Setup projet + Auth + DB + Wallet de base	MVP minimum pour login + wallet
Sprint 2	2 semaines	Dashboard utilisateur + Profil + Historique wallet	MVP avec dashboard et gestion utilisateurs
Sprint 3	2 semaines	Paiement Stripe + Notification + Ledger complet	MVP complet pour transactions + notifications
Sprint 4	2 semaines	Security service + AI Risk service + Monitoring	Release 1 avec sécurité et scoring IA
Sprint 5	2 semaines	Reporting + tests finaux + hardening	Release finale prête production

Table VII.2 – Planification des Sprints / Releases

VII.3 Répartition des rôles SCRUM

Rôle	Nom / Fonction
Product Owner (PO)	AMOR Houssem – responsable des exigences et priorités
Scrum Master (SM)	KHEDHIRI Montaha – organisation, sprint planning, daily stand-up
Développeur Backend	BEN SLIMA Wissem – API .NET 8, microservices, base de données
Développeur Frontend	BAHROUNI Seif – Angular UI, dashboard utilisateur et admin
Développeur IA	AMOR Houssem – modèle IA, scoring de risque, intégration Kafka
DevOps / QA	BEN SLIMA Wissem – CI/CD, Docker, monitoring, tests

Table VII.3 – Rôles SCRUM

VII.4 Choix Technologiques et Outils de Collaboration

- **Frontend :** Angular (TypeScript)
- **Backend :** .NET 8 (C#, architecture microservices)
- **Base de données :** PostgreSQL (utilisateurs, transactions, logs), Redis (cache, sessions, rate-limiting)

- **Messagerie asynchrone :** Apache Kafka (communication inter-services, patron Outbox)
- **IA / ML :** Ollama (Mistral 7B, LLaMA 3) pour le scoring de risque ; Isolation Forest et Autoencoders (scikit-learn, HuggingFace) pour la détection d'anomalies ; CodeLlama pour la détection d'injections SQL
- **Cloud / Infra :** AWS (free tier) ; Docker + Kubernetes (orchestration, scaling) ; Grafana + Prometheus (monitoring) ; Loki (logs centralisés)
- **Outils de collaboration :** GitHub (versioning, PR, CI/CD via GitHub Actions) ; Jira (gestion des sprints et backlog)

VII.5 Développement du Sprint 1 (MVP)

VII.5.1 Objectif Sprint 1

Mise en place du projet + base de données + authentification + wallet minimal.

VII.5.2 Tâches Sprint 1

Tâche	**Responsable**	**Statut**
Initialiser repos Git + branches	Dev 1	☐
Config Docker Compose (PostgreSQL + Redis + Kafka)	DevOps	☐
Endpoint register/login JWT + middleware RBAC	Dev Backend	☐
Création wallet à l'inscription	Dev Backend	☐
Endpoint consultation solde wallet	Dev Backend	☐
Health checks services	DevOps	☐
README + documentation setup local	DevOps	☐

Table VII.4 – Tâches Sprint 1

VII.5.3 Livrables Sprint 1

- Repo GitHub structuré
- Base de données opérationnelle
- Authentification JWT fonctionnelle
- Wallet minimal fonctionnel
- Documentation pour démarrage local

Conclusion Générale

Ce rapport a présenté le projet **SaveSafe Wallet (SSW)**, une plateforme bancaire web sécurisée intégrant un portefeuille numérique et un système de détection de menaces par Intelligence Artificielle.

Nous avons détaillé le contexte et les objectifs du projet, la description fonctionnelle de la plateforme (gestion de comptes, wallet, paiements Stripe, sécurité IA, notifications, reporting), ainsi que l'étude comparative avec les solutions existantes sur le marché. La spécification des besoins fonctionnels et non fonctionnels a permis de cadrer précisément le périmètre du MVP, et l'identification des acteurs a clarifié les interactions attendues avec le système.

La méthodologie SCRUM a été adoptée avec un découpage en cinq sprints de deux semaines. Le product backlog couvre onze EPICs allant du cadrage initial jusqu'aux tests et au hardening de production. Le premier sprint, axé sur le setup projet, l'authentification JWT et le wallet minimal, constitue la base sur laquelle les sprints suivants viendront ajouter les modules de paiement, de sécurité IA, de notification et de reporting.

Les choix technologiques – .NET 8 pour le backend, Angular pour le frontend, PostgreSQL et Redis pour les données, Apache Kafka pour la messagerie, et Ollama pour les modèles IA – garantissent une architecture modulaire, performante et entièrement basée sur des outils open-source.

Les prochaines étapes consisteront à développer les sprints 2 à 5, intégrer les diagrammes d'architecture et de séquence détaillés, et préparer la release de production avec les tests E2E et le hardening sécurité.